BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 3

Roll no: 42 PRN No: 12414728 Name of Student: **Saachi Laturiya**

Subject Name : Deep Learning

Problem Statement : Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm :

1. Import the required Python libraries for data handling, preprocessing, visualization, deep learning, and performance evaluation.
2. Load the Breast Cancer Wisconsin dataset from the Scikit-learn library.
3. Separate the dataset into input features (X) and target labels (y).
4. Split the dataset into training and testing sets.
5. Standardize the training and testing data using StandardScaler.
6. Create a Multilayer Perceptron (MLP) model using TensorFlow/Keras.
7. Compile the model using the Adam optimizer, binary cross-entropy loss function, and accuracy as the evaluation metric.
8. Train the model using different learning rates (0.1, 0.01, and 0.001) while keeping the number of epochs constant.
9. Evaluate the accuracy obtained for each learning rate and visualize the comparison.
10. Train the model using different numbers of epochs (10, 50, and 100) while keeping the learning rate constant.
11. Compare the model accuracy for different epoch values using a graph.
12. Select the best learning rate and epoch combination to train the final MLP model.
13. Predict the class labels for the testing dataset.
14. Evaluate the final model using accuracy, confusion matrix, and classification report.
15. Visualize the training accuracy, training loss, and confusion matrix to analyze the model performance.

Code :

```python
# Import Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Load Dataset
cancer = load_breast_cancer()

X = cancer.data
y = cancer.target

df = pd.DataFrame(X, columns=cancer.feature_names)
df["Target"] = y

print(df.head())

print("\nDataset Shape :", df.shape)

print("\nClass Distribution")
print(df["Target"].value_counts())
```

```python
# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Feature Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Learning Rate Analysis
learning_rates = [0.1, 0.01, 0.001]

lr_accuracy = []

for lr in learning_rates:

    model = Sequential([
        Dense(16, activation='relu', input_shape=(30,)),
        Dense(8, activation='relu'),
        Dense(1, activation='sigmoid')
    ])

    optimizer = Adam(learning_rate=lr)

    model.compile(
        optimizer=optimizer,
        loss='binary_crossentropy',
        metrics=['accuracy']
    )
```

```
    model.fit(
        X_train,
        y_train,
        epochs=50,
        batch_size=16,
        validation_split=0.2,
        verbose=0
    )

    loss, accuracy = model.evaluate(
        X_test,
        y_test,
        verbose=0
    )

    lr_accuracy.append(accuracy)

    print(f"Learning Rate = {lr}   Accuracy = {accuracy:.4f}")

# Visualization 1
# Learning Rate vs Accuracy
plt.figure(figsize=(6,4))

plt.bar(
    [str(i) for i in learning_rates],
    lr_accuracy,
    color="skyblue"
)

plt.xlabel("Learning Rate")
plt.ylabel("Accuracy")
plt.title("Learning Rate vs Accuracy")
```

```
plt.show()

# Epoch Analysis
epochs_list = [10, 50, 100]

epoch_accuracy = []

for ep in epochs_list:

    model = Sequential([
        Dense(16, activation='relu', input_shape=(30,)),
        Dense(8, activation='relu'),
        Dense(1, activation='sigmoid')
    ])

    optimizer = Adam(learning_rate=0.001)

    model.compile(
        optimizer=optimizer,
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    model.fit(
        X_train,
        y_train,
        epochs=ep,
        batch_size=16,
        validation_split=0.2,
        verbose=0
    )
```

```
    loss, accuracy = model.evaluate(
        X_test,
        y_test,
        verbose=0
    )

    epoch_accuracy.append(accuracy)

    print(f"Epochs = {ep}   Accuracy = {accuracy:.4f}")


# Visualization 2
# Epochs vs Accuracy
plt.figure(figsize=(6,4))

plt.plot(
    epochs_list,
    epoch_accuracy,
    marker='o',
    linewidth=2
)

plt.xlabel("Number of Epochs")
plt.ylabel("Accuracy")
plt.title("Epochs vs Accuracy")
plt.grid(True)

plt.show()

# Final Model
final_model = Sequential([
    Dense(16, activation='relu', input_shape=(30,)),
    Dense(8, activation='relu'),
    Dense(1, activation='sigmoid')
```

```
optimizer = Adam(learning_rate=0.001)

final_model.compile(
    optimizer=optimizer,
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history = final_model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=16,
    validation_split=0.2,
    verbose=1
)

# Model Evaluation
loss, accuracy = final_model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\nFinal Test Accuracy:", accuracy)

# Prediction
y_pred = final_model.predict(X_test)

y_pred = (y_pred > 0.5).astype(int)

print("\nAccuracy Score:")
print(accuracy_score(y_test, y_pred))
```

```
print("\nConfusion Matrix:")
cm = confusion_matrix(y_test, y_pred)
print(cm)

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=cancer.target_names
))

# Visualization 3
# Training Accuracy
plt.figure(figsize=(7,5))

plt.plot(
    history.history['accuracy'],
    label='Training Accuracy'
)

plt.plot(
    history.history['val_accuracy'],
    label='Validation Accuracy'
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()

plt.show()
```

```
# Visualization 4
# Training Loss
plt.figure(figsize=(7,5))

plt.plot(
    history.history['loss'],
    label='Training Loss'
)

plt.plot(
    history.history['val_loss'],
    label='Validation Loss'
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training vs Validation Loss")
plt.legend()

plt.show()

# Visualization 5
# Confusion Matrix Heatmap
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=cancer.target_names,
    yticklabels=cancer.target_names
)
```

```
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```

Output :

1. Breast Cancer Wisconsin dataset preview, dataset dimensions, and class distribution.

```
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
0                 0.07871  ...          17.33           184.60      2019.0
1                 0.05667  ...          23.41           158.80      1956.0
2                 0.05999  ...          25.53           152.50      1709.0
3                 0.09744  ...          26.50            98.87       567.7
4                 0.05883  ...          16.67           152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

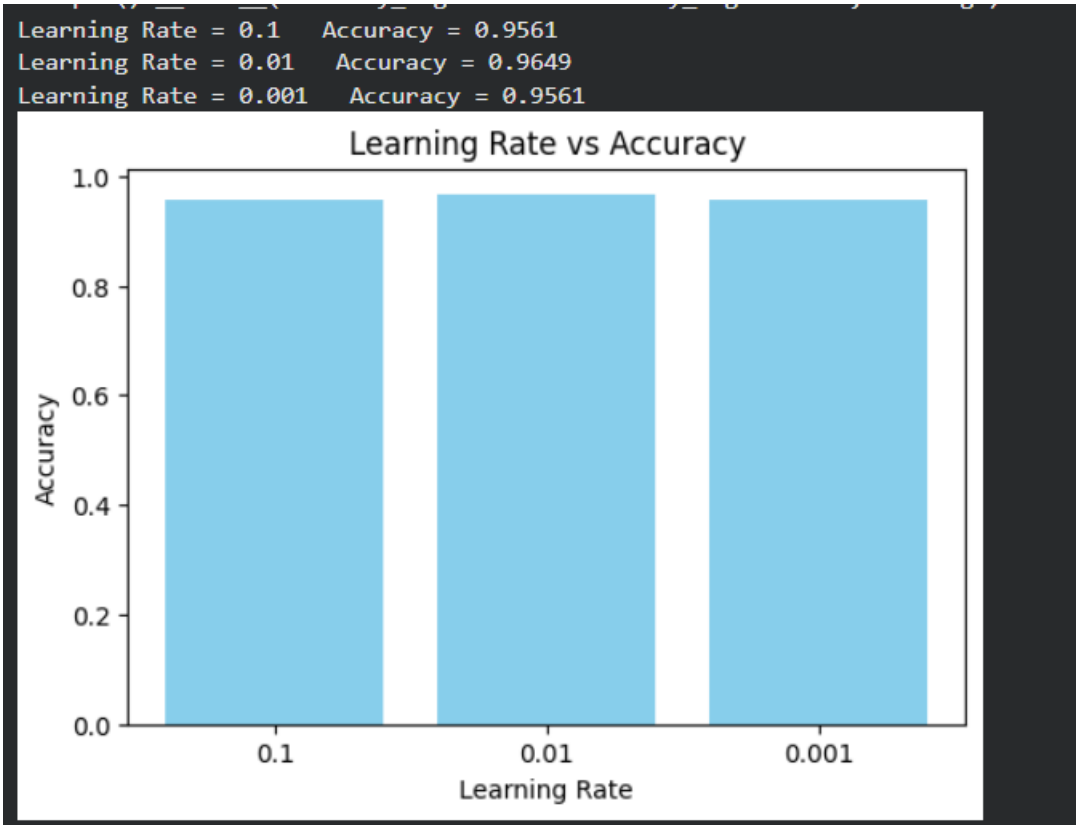
   worst symmetry  worst fractal dimension  Target
0          0.4601                  0.11890       0
1          0.2750                  0.08902       0
2          0.3613                  0.08758       0
3          0.6638                  0.17300       0
4          0.2364                  0.07678       0
```

```
[5 rows x 31 columns]

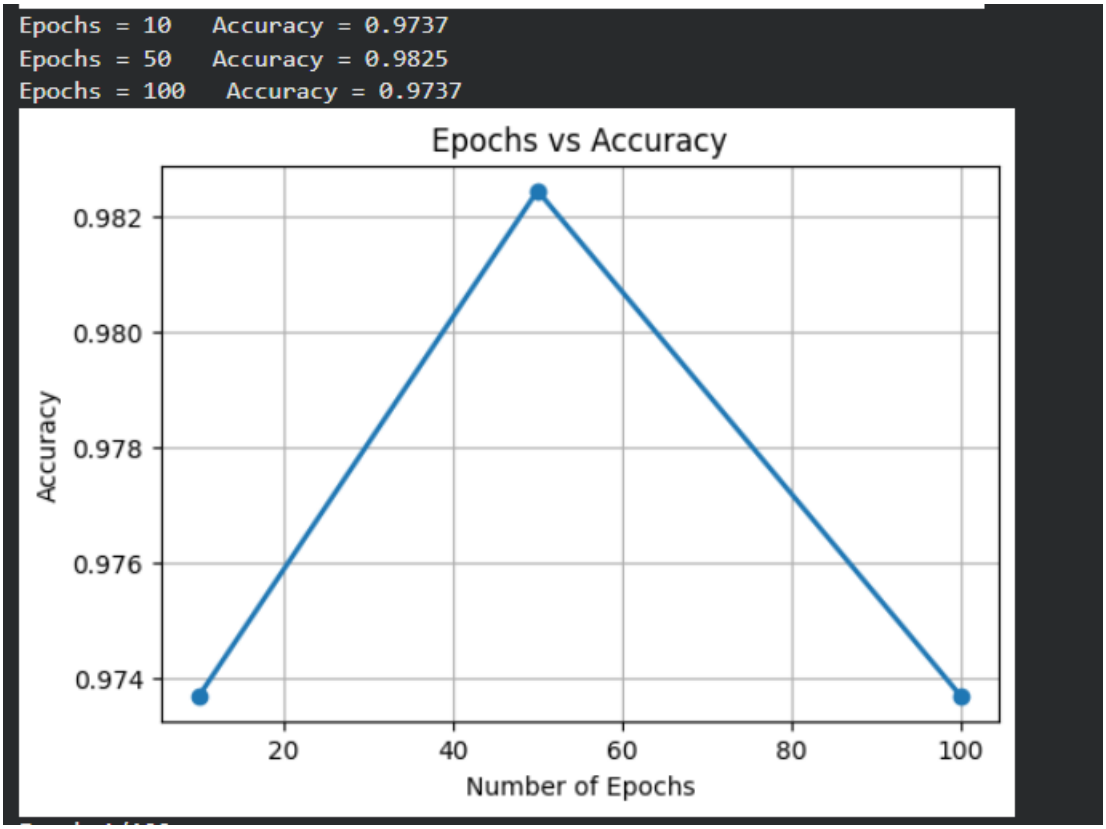
Dataset Shape : (569, 31)

Class Distribution
Target
1    357
0    212
Name: count, dtype: int64
```

2. Effect of different learning rates on the accuracy of the MLP model.



3. Effect of different numbers of epochs on the accuracy of the MLP model.



4. Performance evaluation of the trained MLP model showing accuracy, confusion matrix, and classification report.

```
Final Test Accuracy: 0.9824561476707458
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step

Accuracy Score:
0.9824561403508771

Confusion Matrix:
[[42  1]
 [ 1 70]]

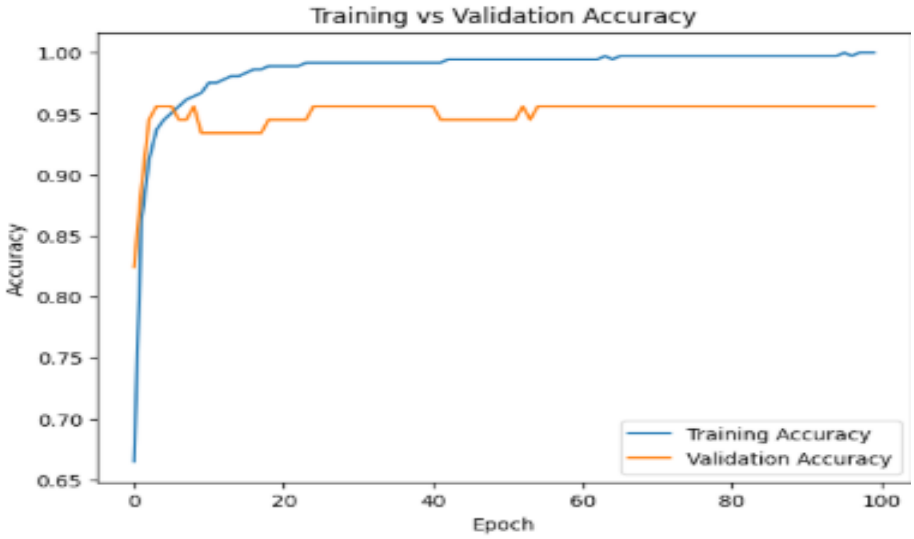
Classification Report:
              precision    recall  f1-score   support

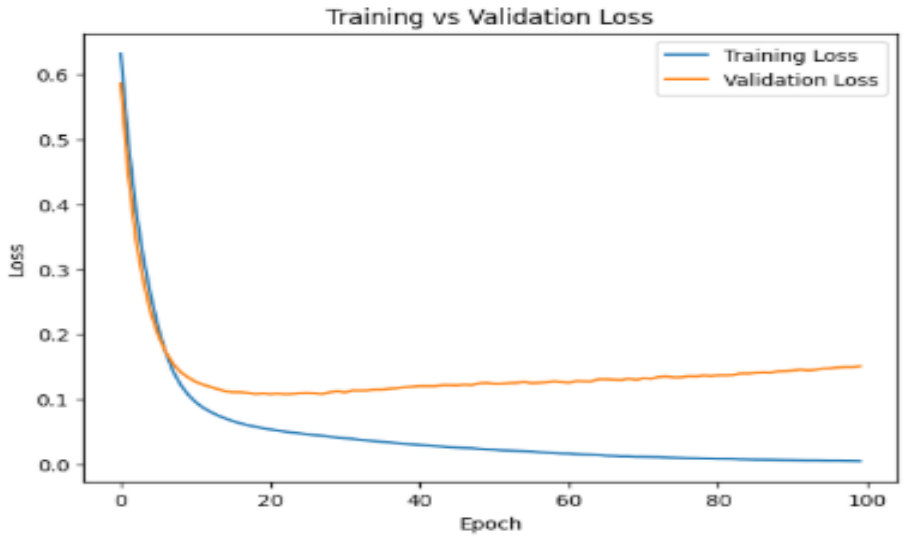
   malignant       0.98      0.98      0.98        43
      benign       0.99      0.99      0.99        71

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114
```

5. Training and validation accuracy of the MLP model over different epochs.

Training and validation loss of the MLP model over different epochs.





6. Confusion Matrix Heatmap

